

1. Transport Layer Core Role & Addressing

L4 Protocol Engine

- Process-to-Process Delivery:** Network layer delivers host-to-host (IP). Transport layer delivers process-to-process using **Port Addresses**.
- Socket Address:** Socket = IP Address + Port Number (e.g., 192.168.1.10:80). Uniquely identifies an active socket endpoint globally.
- Port Range (16-bit: 0 to 65,535):**
 - Well-Known (0–1023):* Assigned by IANA (HTTP:80, HTTPS:443, DNS:53, FTP:20/21, SSH:22, SMTP:25).
 - Registered (1024–49151):* Registered for proprietary apps (MySQL:3306, RDP:3389).
 - Dynamic / Ephemeral (49152–65535):* Assigned temporarily by OS to client processes.
- Multiplexing & Demultiplexing:**
 - Multiplexing (Sender):* Gathers data from multiple sockets, adds L4 headers, and passes down to IP layer.
 - Demultiplexing (Receiver):* Reads Dst Port, routes segment to matching socket buffer.

2. TCP Foundations & Byte-Stream Model

RFC 793

- Core Attributes:** Connection-Oriented, Point-to-Point (unicast only), Reliable (ACK + Retransmit), Full-Duplex (simultaneous 2-way), Stream-Oriented.
- Byte-Stream Service:** TCP treats data as a continuous sequence of individual bytes, NOT discrete packets. Transferred in chunks called **Segments**.
- Circular Buffers:** Send buffer holds bytes waiting for transmission / ACK. Receive buffer reorders out-of-order bytes and absorbs bursts.
- Sequence & ACK Numbers:**
 - Sequence Number (32-bit):* Identifies byte number of 1st data byte in segment: $Seq = ISN + 1st_byte_offset$.
 - ACK Number (32-bit):* Cumulative acknowledgment: tells sender "I have safely received all bytes up to N-1; I expect byte N next".

3. TCP Segment Header Architecture

20 to 60 Bytes

Field Name	Width	Function & Rules
Source / Dest Port	16 b each	Calling & receiving process port numbers.
Sequence Number	32 bits	First data byte's byte-stream number.
Acknowledgment No	32 bits	Next expected byte number (valid if ACK=1).
Header Length (HLEN)	4 bits	No. of 4-byte words. Formula: Bytes = HLEN × 4. Minimum: 5 (20B), Max: 15 (60B).
Reserved	6 bits	Must be 0 (used for NS, CWR, ECE flags).
Control Flags	6 bits	URG, ACK, PSH, RST, SYN, FIN (1 bit each).
Window Size (rwnd)	16 bits	Receiver buffer credit in bytes (Flow control).
Checksum	16 bits	Mandatory! Covers Header + Data + Pseudo-Header.
Urgent Pointer	16 bits	Offset from Seq No to last urgent byte (if URG=1).
Options + Padding	0–40 B	MSS (Max Segment Size), Window Scale, SACK, Timestamps. Padded to 32-bit boundary.

4. TCP 6 Control Flags Summary

1-Bit Triggers

- URG:** Urgent pointer field is valid; segment contains high-priority out-of-band data.
- ACK:** Acknowledgment field is valid (set in all segments post initial SYN).
- PSH:** Receiver must push data immediately to application without buffering.
- RST:** Resets connection abruptly due to error, port mismatch, or host crash.
- SYN:** Synchronizes sequence numbers during connection establishment. Consumes 1 sequence number.
- FIN:** Sender has finished sending data; initiates teardown. Consumes 1 sequence number.

5. Connection Establishment (3-Way Handshake)

SYN-ACK

- Step 1 (Client → Server):** SYN=1, ACK=0, Seq=ISN_c. Client enters SYN-SENT. Consumes 1 Seq No (no payload).
- Step 2 (Server → Client):** SYN=1, ACK=1, Seq=ISN_s, Ack=ISN_c+1. Server allocates buffers/TCB and enters SYN-RCVD. Consumes 1 Seq No.
- Step 3 (Client → Server):** SYN=0, ACK=1, Seq=ISN_c+1, Ack=ISN_s+1. Both enter ESTABLISHED. Can carry data payload.
- SYN Flooding Attack (DoS):** Attacker spams spoofed SYN packets; server creates half-open connections in backlog queue until memory runs out.
- Defense — SYN Cookies:** Server encodes state into $ISN_s = Hash(src, dst, secret)$ without allocating RAM until valid 3rd ACK arrives.

6. Connection Termination & Half-Close

FIN Handshake

- Normal 4-Way Handshake:**
 - Client sends FIN=1, Seq=u → Enters FIN-WAIT-1.
 - Server sends ACK=1, Ack=u+1 → Enters CLOSE-WAIT; Client enters FIN-WAIT-2.
 - Server finishes residual data, sends FIN=1, Seq=v, Ack=u+1 → Enters LAST-ACK.
 - Client sends ACK=1, Ack=v+1 → Enters TIME-WAIT for **2MSL**; Server enters CLOSED.
- Piggybacked 3-Way Teardown:** If Server has no pending data, it combines Step 2 & 3: sends FIN+ACK directly.
- Half-Close:** Client stops sending data but can still receive data indefinitely until server finishes its transmission.
- TIME-WAIT State (2MSL):** Client waits $2 \times \text{MSL}$ (~120s) to:
 - Ensure final ACK reaches server (resend if lost).
 - Allow duplicate / lingering segments to expire in internet routers before reuse.

7. TCP Data Transfer, PUSH & URGENT Mechanics

Stream Control

- Normal Data Stream:** Segment carries data bytes. Sequence number is the index of first byte. Acknowledgment number confirms receipt of previous bytes.
- Pushing Data (PSH Flag):**
 - Normal behavior: TCP sender/receiver buffers data for transmission efficiency.
 - With `PSH=1`: Sender flushes output buffer instantly; Receiver flushes input buffer directly to app layer without waiting for a full MSS block.
- Urgent Data (URG Flag & Pointer):**
 - Used for out-of-band signals (e.g., Telnet/SSH Ctrl+C, abort commands).
 - Urgent Pointer = 16-bit offset added to Sequence Number pointing to byte immediately after last urgent byte.

8. Flow Control & Credit Sliding Window

Buffer Matching

- Core Objective:** Prevent fast sender from overflowing slow receiver's buffer.
- Effective Window Formula:**

Effective Window = min(rwnd, cwnd)

rwnd = Receiver Advertised Window; cwnd = Congestion Window.
- Window States (Sender Perspective):**
 - Opening: Right wall expands rightward (receiver advertises larger buffer).
 - Closing: Left wall slides rightward (ACK acknowledges sent bytes).
 - Shrinking: Right wall moves leftward (strongly discouraged by RFC 793).
- Zero-Window Deadlock:** Receiver advertises rwnd=0. Later, buffer frees up and receiver sends rwnd=2000 update, which gets LOST in transit! Both sides wait indefinitely!
- Deadlock Solution (Persistence Timer):** When rwnd=0, sender starts Persistence Timer. On expiry, sends a 1-byte **Window Probe Segment**. Receiver replies with current rwnd.
- Silly Window Syndrome (SWS):**
 - Receiver-driven: Receiver advertises tiny 1-byte window. Fixed by **Clark's Solution**: Do not advertise window until $rwnd \geq \min(\text{MSS}, \text{Buffer}/2)$.
 - Sender-driven: App writes 1 byte at a time. Fixed by **Nagle's Algorithm**: Send 1st byte immediately; buffer subsequent tiny bytes until full MSS or previous ACK arrives.

9. TCP Error Control & Retransmission Engine

Reliability

- Three Error Detection Mechanisms:**
 - Checksum: 16-bit 1s complement sum over header, data, pseudo-header. If corrupt, dropped silently.
 - Acknowledgments: Cumulative ACKs. Confirms all contiguous bytes up to ACK-1.
 - Retransmission Timer (RTO): Resends segment if no ACK received before RTO expires.
- Retransmission Triggers:**
 - RTO Timeout: Heavy loss/congestion indicator $\implies$ drops cwnd = 1 MSS.
 - 3 Duplicate ACKs (Fast Retransmit): Segment lost but network still delivering packets $\implies$ resends missing segment without waiting for RTO timer.
- Out-of-Order Segments:** TCP does not drop out-of-order bytes; it buffers them and sends duplicate ACK of last contiguous in-order byte.

10. Dynamic RTO Formulas (Jacobson & Karn)

Mathematical Core

- Sample RTT:** Time measured between sending segment and receiving ACK.
- Smoothed RTT (SRTT):** Weighted moving average:

$$SRTT_{\text{new}} = (1 - \alpha) \cdot SRTT_{\text{old}} + \alpha \cdot \text{SampleRTT}$$
$$\quad (\alpha = 0.125)$$
- RTT Variation (RTTVAR):** Jitter/variance metric:

$$RTTVAR_{\text{new}} = (1 - \beta) \cdot RTTVAR_{\text{old}} + \beta \cdot |\text{SampleRTT} - \text{SampleRTT}|$$
$$\quad (\beta = 0.25)$$
- Retransmission Timeout (RTO):**

RTO = SRTT + 4 · RTTVAR
- Karn's Algorithm & Exponential Backoff:**
 - Rule 1: NEVER update SampleRTT for retransmitted segments (ACK ambiguity).
 - Rule 2 (Timer Backoff): On retransmission, double RTO: $RTO_{\text{new}} = 2 \times RTO_{\text{old}}$ (exponential backoff up to 64s). Reset to normal formula only after a fresh, non-retransmitted ACK.

11. Flow Control vs Congestion Control Distinction

Key Difference

Feature	Flow Control	Congestion Control
Target Entity	Protects Receiver Buffer	Protects Global Network Routers
Governed By	Advertised Window (rwnd)	Congestion Window (cwnd)
Metric Origin	Receiver feedback (ACK field)	Packet loss & delay inferred by sender
Equation	Actual Transmission Window = min(rwnd, cwnd)	

12. TCP Congestion Control Algorithms

AIMD Architecture

- Additive Increase Multiplicative Decrease (AIMD):** Increases bandwidth gently; cuts drastically on overload to maintain network stability.
- Phase 1: Slow Start (\$cwnd < ssthresh\$):**
 - Initial $cwnd = 1 \text{ MSS}$.
 - For every ACK received: $cwnd = cwnd + 1 \text{ MSS}$ $\implies$ Doubles every RTT (2^n exponential growth).
 - Transitions to Congestion Avoidance when $cwnd \geq ssthresh$.
- Phase 2: Congestion Avoidance (\$cwnd \geq ssthresh\$):**
 - Increases by 1 MSS per RTT: $cwnd = cwnd + (1 / cwnd)$ per ACK $\implies$ Linear growth.
- Loss Detection & Recovery:**
 - Case A: RTO Timeout (Severe Congestion):**
 $ssthresh = cwnd / 2$, $cwnd = 1 \text{ MSS}$ $\implies$ Restart Slow Start.
 - Case B: 3 Duplicate ACKs (Mild Congestion / Isolated Drop):**
TCP Tahoe: $ssthresh = cwnd / 2$, $cwnd = 1 \text{ MSS}$ (Restart Slow Start).
TCP Reno: $ssthresh = cwnd / 2$, $cwnd = ssthresh + 3 \text{ MSS}$ (Fast Recovery $\implies$ skips Slow Start).

13. TCP Timers Architecture & 11-State FSM

RFC 793

- Four Operational Timers:**
 - RTO Timer:** Dynamic retransmission timer per unacknowledged segment.
 - Persistence Timer:** Prevents zero-window deadlock; sends periodic probes.
 - Keepalive Timer:** 2 hours idle detection. Sends 10 probes @ 75s intervals before resetting.
 - TIME-WAIT Timer:** Runs for 2MSL (~120s) before client socket is recycled.
- TCP 11 States:** CLOSED, LISTEN, SYN-SENT, SYN-RCVD, ESTABLISHED, FIN-WAIT-1, FIN-WAIT-2, CLOSE-WAIT, CLOSING, LAST-ACK, TIME-WAIT.

14. User Datagram Protocol (UDP RFC 768)

Lightweight

- Properties:** Connectionless, Unreliable, Message-Oriented (preserves application message boundaries), No Congestion Control, Minimal Overhead.
- 8-Byte Header:** Src Port (16b) | Dst Port (16b) | Length (16b) | Checksum (16b).
- Length Field:** Header (8B) + Data bytes. Min value = 8.
- Pseudo-Header (12 Bytes):** IP Source (4B), IP Dest (4B), Zero (1B), Protocol (1B = 17), UDP Length (2B). Used exclusively during 1s-complement checksum computation to detect cross-host delivery errors. Checksum is optional in IPv4 (0 if disabled), mandatory in IPv6.

15. Master Comparison: TCP vs UDP

Exam Favorite Table

Parameter	TCP (RFC 793)	UDP (RFC 768)
Connection	Connection-Oriented (3-way)	Connectionless
Reliability	Reliable (ACK + Retransmit)	Unreliable (Best Effort)
Header Size	20–60 Bytes (Heavy)	8 Bytes fixed (Lightweight)
Data Delivery	Byte Stream (No boundaries)	Message / Datagram-oriented
Flow / Congestion	Yes (Sliding window + AIMD)	None (Blasts at app rate)
Speed / Latency	Slower (Handshake overhead)	Ultra-fast, lowest latency
Communication	Unicast point-to-point only	Unicast, Multicast & Broadcast
Key Applications	HTTP/S, FTP, SMTP, SSH	DNS, DHCP, VoIP, Gaming, RTP

16. Quality of Service (QoS) & Traffic Shaping

L4 Optimization

- 4 QoS Metrics:** Delay/Latency (propagation+queuing), Jitter (delay variance), Bandwidth/Throughput, Packet Loss rate.
 - Packet Scheduling Disciplines:**
 - FIFO:** First-in first-out; zero fairness, prone to bufferbloat.
 - Priority Queuing:** High-priority traffic served first; can starve low-priority.
 - Weighted Fair Queuing (WFQ):** Allocates guaranteed bandwidth ratio per queue.
 - Leaky Bucket vs Token Bucket Traffic Shapers:**
 - Leaky Bucket:** Eliminates bursts; outputs at a rigid constant rate. Overflows drop packets.
 - Token Bucket:** Generates tokens at rate \$R\$. Allows bursts up to capacity \$C\$ at maximum rate \$M\$.
- Max Burst Duration:** $T = C / (M - R)$
 C = Token Capacity, M = Max Burst Transmission Rate, R = Token Arrival Rate.

17. AKTU Numerical & Formula Cheat Bank

High-Yield Formulas

- TCP Header Length (Bytes) = $HLEN \times 4$. If $HLEN = 0111_2$ (7) $\implies 7 \times 4 = 28$ Bytes (20B base + 8B options).
- $MSS = MTU - (IP_Header + TCP_Header) = 1500 - 40 = 1460$ Bytes.
- Slow Start Rounds to reach Threshold: $n = \log_2(ssthresh / cwnd_0)$.
- Window Utilization / Efficiency: $\eta = \frac{W}{cwnd} \cdot \text{MSS}$ $\implies \frac{\text{Bandwidth} \times \text{RTT}}$.
- Token Bucket Burst: $\text{Max Volume Transmitted} = C + R \cdot T = M \cdot T$.